```
            return true;
        } else {
            return isPrime(num, divider + 1);
        }
    }

    public static boolean isPrime(int num) {
        return isPrime(num, 2);
    }

    public static void main(String[] args) {
        for (int i = 0; i < 100; i++) {
            System.out.println(i + " " + isPrime(i));
        }
    }
}
```

Κώδικας 9.12 *Αναδρομική αξιολόγηση πρώτου αριθμού*

Καταρχάς, στον κώδικα 9.12, στη συνάρτηση isPrime(int, int), ελέγχουμε αν ο υπό εξέταση αριθμός είναι μικρότερος ή ίσος από το 2, οπότε επιστρέφουμε false. Στη συνέχεια προκύπτουν δύο βασικές περιπτώσεις. Στην πρώτη περίπτωση, ο num διαιρείται ακριβώς με τον divider οπότε ο num δεν είναι πρώτος και επιστρέφουμε false. Στη δεύτερη βασική περίπτωση, ελέγχουμε αν ο divider υπερβαίνει το num/2. Σε αυτήν την περίπτωση έχουν ελεγχθεί όλοι οι ακέραιοι από 2 έως num/2 και κανένας δεν βρέθηκε να διαιρεί ακριβώς τον num, επομένως ο num είναι πρώτος και επιστρέφουμε true. Αν καμία από τις βασικές περιπτώσεις δεν προκαλέσει επιστροφή της isPrime, τότε προχωρούμε στην αναδρομική κλήση αυξάνοντας τον divider κατά 1.

Παρόμοια με την προηγούμενη άσκηση, ο divider ως παράμετρος εξυπηρετεί στην αναδρομική κλήση. Ο χρήστης της συνάρτησής μας δεν χρειάζεται να γνωρίζει καν την ύπαρξή του. Έτσι έχουμε κάνει την isPrime(int, int) private και παρέχουμε κατάλληλο περιτύλιγμα, την isPrime(int) που είναι public.

9.6.4 Selection Sort

Να αναπτυχθεί αναδρομική συνάρτηση ταξινόμησης μονοδιάστατου πίνακα ακέραιων με βάση τον αλγόριθμο selection sort (άσκηση 6.5.1). Να κατασκευαστεί κατάλληλος κώδικας ελέγχου με χρήση της Junit.

Λύση

Τη λογική του selection sort έχουμε ήδη παρουσιάσει στην άσκηση 6.5.1. Στον κώδικα 9.13 δίνουμε την αναδρομική και δομημένη υλοποίησή της.

```
import java.util.Arrays;

public class SelectionSort {//Κώδικας 9.13

    private static void swap(int[] array, int idx1, int idx2) {
        int i = array[idx1];
        array[idx1] = array[idx2];
        array[idx2] = i;
    }

    private static int idxOfMin(int[] array, int startIdx) {
        int min = array[startIdx];
        int rVal = startIdx;
        for (int i = startIdx + 1; i < array.length; i++) {
            if (array[i] < min) {
```

```
                min = array[i];
                rVal = i;
            }
        }
        return rVal;
    }

    private static void sort(int[] array, int startIdx) {
        int posOfMin = idxOfMin(array, startIdx);
        if (posOfMin != startIdx) {
            swap(array, posOfMin, startIdx);
        }
        startIdx++;
        if (startIdx < array.length - 1) {
            sort(array, startIdx);
        }
    }

    public static void sort(int[] array) {
        sort(array, 0);
    }

    public static void main(String[] args) {
        int[] tbl = {2, 1, 8, 6, 5, 4};
        sort(tbl);
        System.out.println(Arrays.toString(tbl));
    }
}
```

Κώδικας 9.13 *Αναδρομική υλοποίηση της selection sort*

Στον κώδικα 9.13 η αναδρομική υλοποίηση της selection sort δίνεται από τη συνάρτηση sort(int[], int). Η δεύτερη παράμετρος προκύπτει για διευκόλυνση της αναδρομικής κλήσης. Παρόμοια με τις δύο προηγούμενες ασκήσεις παρέχεται μια συνάρτηση-περιτύλιγμα, η sort(int[]). Οι υπόλοιπες συναρτήσεις είναι γνωστές καθώς έχουν ήδη συζητηθεί.

Ας δούμε λοιπόν την sort(int[], int). Η παράμετρος startIdx καθορίζει τη θέση του πίνακα από την οποία και μετά, ο πίνακας είναι αταξινόμητος. Αρχικά, η συνάρτηση εντοπίζει τη θέση του μικρότερου ακέραιου στο υπό ταξινόμηση τμήμα του πίνακα, δηλαδή από το startIdx και μετά. Αν το μικρότερο στοιχείο βρίσκεται μετά τη θέση startIdx, τότε ανταλλάσσει το μικρότερο στοιχείο με το στοιχείο στη θέση startIdx. Στη συνέχεια, αυξάνει το startIdx κατά 1 έτσι ώστε να συνεχίσει η ταξινόμηση από την επόμενη θέση στον πίνακα. Αν το startIdx δείχνει πριν την τελευταία θέση, σημαίνει ότι υπολείπεται τμήμα του πίνακα προς ταξινόμηση, οπότε γίνεται η αναδρομική κλήση, διαφορετικά τερματίζεται η αναδρομική κλήση και ο πίνακας είναι ταξινομημένος.

Στη συνέχεια παρουσιάζουμε τον κώδικα ελέγχου.

```
public class SelectionSortTest {

    public SelectionSortTest() {
    }

    /**
     * Test of sort method, of class SelectionSort.
     */
    @Test
    public void testSort() {
        System.out.println("sort");
```

```
        Random gen = new Random();
        for (int i = 0; i < 100; i++) {
            int[] array = new int[gen.nextInt(100) + 10];
            for (int j = 0; j < array.length; j++) {
                array[j] = gen.nextInt();
            }
            SelectionSort.sort(array);
            for (int j = 0; j < array.length - 1; j++) {
                if (array[j + 1] < array[j]) {
                    fail();
                }
            }
        }
    }

}
```

Κώδικας 9.14 *Κώδικας ελέγχου της SelectionSort.sort*

Στον κώδικα 9.14 κατασκευάζουμε 100 πίνακες με μέγεθος μεταξύ 10 και 110. Στη συνέχεια εκχωρούμε σε κάθε θέση ενός πίνακα έναν τυχαίο ακέραιο. Σημειώστε πως η nextInt χωρίς παραμέτρους μπορεί να επιστρέψει οποιαδήποτε ακέραιη τιμή. Αφού γεμίσουμε τον πίνακα με τυχαίες τιμές, προχωρούμε στην ταξινόμησή του. Τέλος, ελέγχουμε αν ο πίνακας είναι σωστά ταξινομημένος. Σε περίπτωση που εντοπίσουμε μικρότερο στοιχείο του πίνακα να έπεται μεγαλύτερου, καλούμε την fail και σημειώνουμε το λάθος.

9.6.5 Δυαδική αναζήτηση

Να αναπτυχθεί αναδρομική συνάρτηση δυαδικής αναζήτησης σε μονοδιάστατο πίνακα ακέραιων. Να κατασκευαστεί κατάλληλος κώδικας ελέγχου με χρήση της Junit.

Λύση
Τον αλγόριθμο δυαδικής αναζήτησης τον έχουμε ήδη συζητήσει και έχουμε δώσει και την μη αναδρομική υλοποίησή του (ενότητα 6.5.2). Έτσι, εδώ θα συζητήσουμε αποκλειστικά την αναδρομική υλοποίησή του.

```
public class BinarySearch {

    private static int binarySearch(int[] tbl, int schElement, int from, int
to) {
        if (from <= to) {
            int mid = from + (to - from) / 2;
            if (tbl[mid] == schElement) {
                return mid;
            }
            if (schElement < tbl[mid]) {
                return binarySearch(tbl, schElement, from, mid - 1);
            }
            return binarySearch(tbl, schElement, mid + 1, to);
        }
        return -1;
    }

    public static int binarySearch(int[] tbl, int schElement) {
        return binarySearch(tbl, schElement, 0, tbl.length - 1);
    }
```

```
}
```

Κώδικας 9.15 *Αναδρομική δυαδική αναζήτηση*

Στον κώδικα 9.15 η binarySearch(int[], int, int, int) είναι η αναδρομική συνάρτηση. Οι παράμετροι from και to έχουν προστεθεί για να διευκολύνουν την αναδρομή. Η συνάρτηση-περικάλυμμα καλεί την αναδρομική με παραμέτρους 0 και tbl.length-1 ώστε η αναζήτηση να αφορά καταρχάς όλον τον πίνακα. Μέσα στην αναδρομική συνάρτηση γίνεται έλεγχος εάν το from είναι μικρότερο από το to, δηλαδή αν υπάρχει τμήμα του πίνακα που δεν έχει ελεγχθεί. Αν υπάρχει τέτοιο τμήμα, ενημερώνεται κατάλληλα η mid που δείχνει στο μέσο του υπό αναζήτηση τμήματος. Στη συνέχεια ελέγχεται αν το μέσον έχει την τιμή αναζήτησης. Αν ναι, η συνάρτηση επιστρέφει το μέσον ως τη θέση του στοιχείου που αναζητούμε. Αν όχι, η συνάρτηση προχωράει σε αναδρομική κλήση. Αν η τιμή του στοιχείου αναζήτησης είναι μικρότερη από την τιμή στο μέσο, η αναδρομική κλήση αφορά το πρώτο μισό του υπό εξέταση τμήματος, διαφορετικά αφορά το δεύτερο μισό.
Στη συνέχεια δίνουμε τον κώδικα ελέγχου.

```
import java.util.Random;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 *
 * @author Lefteris Moussiades
 */
public class BinarySearchTest {

     /**
     * Test of binarySearch method, of class BinarySearch.
     */
    @Test
    public void testBinarySearch() {
        System.out.println("binarySearch");
        Random gen=new Random();
        for (int i=0; i<100; i++) {
            int[] array=new int[gen.nextInt(100)+10];
            for (int j=0; j<array.length; j++) {
                array[j]=gen.nextInt();
            }
            SelectionSort.sort(array);
            int idx=gen.nextInt(array.length);
            idx=idx==array.length?array.length-1:idx;
            int schElement=array[idx];
            int result = BinarySearch.binarySearch(array, schElement);
            assertTrue(array[result]==schElement);
        }
    }
}
```

Κώδικας 9.16 *Έλεγχος της binarySearch*

Στον κώδικα 9.16 κατασκευάζουμε 100 πίνακες ακέραιων με τυχαίο μήκος από 10 έως 110 και τους γεμίζουμε με τυχαίες ακέραιες τιμές. Στη συνέχεια ταξινομούμε κάθε πίνακα και επιλέγουμε την τιμή ενός τυχαίου στοιχείου του. Τέλος καλούμε την binarySearch ώστε να αναζητήσουμε το στοιχείο που γνωρίζουμε ότι όντως υπάρχει στον πίνακα.

9.7 Ασκήσεις προς λύση

1. Να αναπτυχθεί συνάρτηση που υπολογίζει αναδρομικά τον Μέγιστο Κοινό Διαιρέτη σειράς ακέραιων. Να αναπτυχθεί κατάλληλος κώδικας ελέγχου.

2. Να αναπτυχθεί αναδρομική συνάρτηση static void prtSquareRoots(int i, int n) που τυπώνει τη σειρά των τετραγώνων των αριθμών από i έως n.

3. Να αναπτυχθεί αναδρομική συνάρτηση static void prtSquareRoots(int i, int n) που τυπώνει τη σειρά των τετραγώνων των αριθμών από i έως n, στην ίδια γραμμή, χωρισμένα με κόμμα και διάστημα. Μετά την εκτύπωση της σειράς, η συνάρτηση να προετοιμάζει ώστε η επόμενη έξοδος να βγαίνει στην επόμενη γραμμή. Για παράδειγμα, τα τετράγωνα των αριθμών από 0 έως 9 να τυπωθούν ως εξής:

```
0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

4. Να αναπτύξετε αναδρομική συνάρτηση υπολογισμού του αθροίσματος

$$\sum_{i=1}^{n} \frac{1}{i}$$

5. Πόσες αναδρομικές κλήσεις προκαλεί η κλήση fibonacci(9);

   Υπόδειξη: Τοποθετήστε την αναδρομική Fibonacci σε μία κλάση. Ορίστε στην κλάση μια ακέραια στατική μεταβλητή cnt. Αυξήστε την cnt κατάλληλα μέσα στην Fibonacci. Εμφανίστε το αποτέλεσμα.

6. Να αναπτυχθεί αναδρομική συνάρτηση που επιστρέφει τη μέγιστη τιμή ενός πίνακα ακέραιων.

7. Να αναπτυχθεί αναδρομική συνάρτηση που μετράει τους πεζούς χαρακτήρες σε έναν πίνακα χαρακτήρων.

8. Πόσες αναδρομικές κλήσεις απαιτεί η λύση των πύργων του Ανόι για 10 δίσκους;

9. Να αναπτυχθεί αναδρομική συνάρτηση που εμφανίζει όλες τις αναδιατάξεις (permutations) μιας συμβολοσειράς. Για παράδειγμα, έστω η συμβολοσειρά can, οι δυνατές αναδιατάξεις είναι can, cna, acn, anc, nca, nac.

10. Να αναπτυχθεί αναδρομική συνάρτηση που λαμβάνει ως παραμέτρους δύο συμβολοσειρές και εξετάζει αν η δεύτερη περιέχεται στην πρώτη.

11. Ένας τρόπος για τον υπολογισμό της τετραγωνικής ρίζας ενός πραγματικού αριθμού, x>0, έχει ως εξής: Έστω y μια τυχαία λύση. Αν το y υψωμένο στο τετράγωνο προσεγγίζει ικανοποιητικά το x, τότε η τετραγωνική ρίζα του x είναι το y, διαφορετικά αντικαθιστούμε στη θέση του y, το (y+x/y)/2. Υλοποιήστε αναδρομική συνάρτηση υπολογισμού της τετραγωνικής ρίζας πραγματικού αριθμού.

Βιβλιογραφία

[1] “Chapter 2. The Structure of the Java Virtual Machine.” https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html#jvms-2.5 (accessed Oct. 15, 2021).

[2] E. W. Weisstein, “Hofstadter Male-Female Sequences.” https://mathworld.wolfram.com/HofstadterMale-FemaleSequences.html (accessed Sep. 14, 2021).